AWS Cloud — Issues, Root Causes &

Resolutions: Master Interview Guide

Prepared for: Pushparaj Naik | AWS Cloud & DevOps Architect | 22+ Years Experience
Scope: AWS Infrastructure · Networking · Compute · EKS/Kubernetes · DevOps · DevSecOps · IaC · Observability
Purpose: Interview-Ready Reference — Covers real-world issues encountered across all layers of AWS solutioning

---

Table of Contents

---

1. AWS Networking & VPC Issues

Issue 1.1 — EC2 instance in private subnet cannot reach the internet

Attribute	Detail
Layer	VPC Networking
Symptom	`yum update` times out; instance can't pull packages or container images
Root Cause	No NAT Gateway in the public subnet, or private route table does not have a default route (0.0.0.0/0) pointing to the NAT Gateway

Resolution:

1. Verify a NAT Gateway exists in a **public subnet** (not a private one)
2. Check the route table associated with the private subnet:

```
Destination   Target
0.0.0.0/0  →  nat-<id>   (must exist)
10.0.0.0/16 → local
```

3. Ensure the NAT Gateway has an Elastic IP attached
4. Verify the Internet Gateway is attached to the VPC and the **public subnet's** route table has `0.0.0.0/0 → igw-<id>`

Prevention: Always provision one NAT Gateway per AZ (not just one total) for HA. Use Terraform's `count` or `for_each` pattern tied to AZ count.

Issue 1.2 — EC2 cannot connect to RDS in private subnet (connection refused / timeout)

Attribute	Detail
Layer	Security Groups / NACLs
Symptom	Application logs `Connection refused to db-endpoint:5432`; direct `nc -zv` from EC2 also fails

Attribute	Detail
Root Cause	Security group on RDS does not allow inbound port 5432 from the EC2 security group; OR the NACL on the DB subnet blocks the connection

Systematic troubleshooting (layer by layer):

```
Step 1 — Security Groups:
  ✦ EC2 SG → outbound rule allows port 5432 to RDS SG? (usually all outbound
allowed)
  ✦ RDS SG → inbound rule allows TCP 5432 from EC2 SG ID (NOT CIDR)?

Step 2 — NACLs (stateless — must check both directions):
  ✦ EC2 subnet NACL → outbound: allow TCP 5432 to DB subnet CIDR
  ✦ DB subnet NACL  → inbound: allow TCP 5432 from App subnet CIDR
  ✦ DB subnet NACL  → outbound: allow ephemeral ports 1024–65535 back to app
subnet
  ✦ EC2 subnet NACL → inbound: allow ephemeral ports 1024–65535 from DB
subnet

Step 3 — Route Tables:
  ✦ Both subnets must have "10.0.0.0/16 → local" to route within the VPC

Step 4 — RDS Configuration:
  ✦ RDS must NOT be "Publicly accessible = true" if in private subnet
  ✦ RDS subnet group must include the subnets where nodes are
```

Golden Rule: Always use **Security Group references** (SG ID), not CIDR ranges, for internal SG rules. CIDR ranges break when subnets are added.

Issue 1.3 — ALB returns 502 Bad Gateway

Attribute	Detail
Layer	Load Balancing / Application
Symptom	ALB returns HTTP 502 to clients
Root Cause (multiple)	(a) Target instances are unhealthy (health checks failing) → ALB has no healthy targets; (b) Application on port 8080 is not responding or returning 5xx; (c) Security group on EC2/container does not allow inbound from ALB SG on the app port

Resolution Decision Tree:

```
502 on ALB?
  ├── Check Target Group health in console
  │     ├── All targets "unhealthy" → Check app health check path (/health)
  │     │                           → Check app is listening on the correct
port
  │     │                           → Check EC2 SG allows ALB SG on app port
  │     └── Some targets "draining" → Previous deployment still draining
  │
  └── Check ALB access logs (S3) for backend error details
      ├── `target_status_code: 500` → App throwing 500, fix app
      └── `target_status_code: -` → ALB couldn't connect at all → SG/port
issue
```

Fix for SG: Add inbound rule to EC2/ECS task security group:

```
From: ALB security group ID
Port: 8080 (app port)
Protocol: TCP
```

Issue 1.4 — VPC DNS resolution not working (internal hostnames fail to resolve)

Attribute	Detail
Layer	VPC DNS
Symptom	`nslookup db.internal` or RDS endpoint DNS fails; services can't find each other
Root Cause	`enableDnsSupport` or `enableDnsHostnames` disabled on the VPC; OR Route 53 Private Hosted Zone not associated with the VPC

Resolution:

```
# Check VPC DNS settings
aws ec2 describe-vpc-attribute --vpc-id vpc-xxx --attribute enableDnsSupport
aws ec2 describe-vpc-attribute --vpc-id vpc-xxx --attribute
enableDnsHostnames
# Both must be "true"

# For Private Hosted Zone: associate the zone with the VPC
aws route53 associate-vpc-with-hosted-zone \
```

```
  --hosted-zone-id /hostedzone/XXXXX \
  --vpc VPCRegion=us-east-1,VPCId=vpc-xxx
```

Terraform:

```
resource "aws_vpc" "main" {
  enable_dns_support   = true   # Must be true
  enable_dns_hostnames = true   # Must be true for RDS/other to get DNS
names
}
```

Issue 1.5 — Security Group rule change not taking effect immediately

Attribute	Detail
Layer	Security Groups
Symptom	Added inbound rule to SG but connection still fails
Root Cause	Security groups are **stateful** — rule changes apply immediately to new connections, but ongoing connections established before the change may still be affected. More commonly: the wrong SG was edited, or the instance has multiple SGs

Resolution:

1. Verify the change is on the correct SG (instances can have multiple)
2. Check all SGs attached to the instance — ALL must permit the traffic
3. For outbound initiated from instance: check the **destination SG's inbound rules**
4. Test: Use VPC Reachability Analyzer to trace path end-to-end

```
# Find all SGs on an instance
aws ec2 describe-instances --instance-ids i-xxx \
  --query 'Reservations[].Instances[].SecurityGroups'
```

Issue 1.6 — NACL blocking traffic (intermittent connections, works sometimes)

Attribute	Detail
Layer	NACLs (Stateless Firewall)
Symptom	HTTP requests randomly fail or timeout; TCP SYN gets through but response doesn't
Root Cause	NACL rules are **stateless** — you must explicitly allow return traffic (ephemeral ports 1024-65535). A missing outbound ephemeral port rule on the destination subnet causes the response to be dropped

Resolution — NACL must be bidirectional:

```
# Server subnet NACL (e.g., DB subnet):
Inbound:  Rule 100 — Allow TCP 5432 from App subnet (10.0.10.0/24)
Outbound: Rule 100 — Allow TCP 1024–65535 to App subnet (10.0.10.0/24)  ← OFTEN MISSED

# Client subnet NACL (e.g., App subnet):
Outbound: Rule 100 — Allow TCP 5432 to DB subnet (10.0.20.0/24)
Inbound:  Rule 100 — Allow TCP 1024–65535 from DB subnet (10.0.20.0/24)  ← OFTEN MISSED
```

Interview Insight: NACLs are evaluated in **rule number order** (lowest first). A Rule 90 DENY overrides a Rule 100 ALLOW. SGs evaluate ALL rules simultaneously — there's no ordering.

---

2. Hybrid Connectivity Issues

Issue 2.1 — Site-to-Site VPN tunnel is down or flapping

Attribute	Detail
Layer	VPN / Hybrid Connectivity

Attribute	Detail
Symptom	On-premises → AWS connectivity intermittent; VPN shows one tunnel UP, one DOWN
Root Cause	IKEv2 Phase 1 or Phase 2 parameter mismatch; Dead Peer Detection (DPD) timeout; BGP session not established

Resolution:

```
1. Verify both VPN tunnels are Up (AWS creates 2 tunnels for HA)
   → AWS Console → VPN Connections → Tunnel Status

2. Confirm IKE/IPSec parameters match on both sides:
   ✦ IKE version: IKEv2 (recommended)
   ✦ Encryption: AES-256-GCM
   ✦ Integrity: SHA-256
   ✦ DH Group: 14, 19, 20, or 21
   ✦ Phase 2 (ESP): AES-256-GCM, SHA-256

3. Check CloudWatch VPN metrics:
   ✦ TunnelState = 1 (UP), 0 (DOWN)
   ✦ TunnelDataOut = bytes flowing (should be > 0 for active connections)

4. For BGP issues:
   → Verify ASN configuration matches
   → Check route advertisement (customer gateway must advertise on-prem
CIDR)
   → BGP keepalive timer: default 10s; hold timer: 30s
```

HA Best Practice: AWS creates 2 tunnels per VPN connection. Route both via BGP with different MED values so primary is preferred but failover is automatic.

Issue 2.2 — Direct Connect NOT encrypting traffic (common misconception)

Attribute	Detail
Layer	Direct Connect Security
Symptom	Compliance audit flags that DX traffic is unencrypted despite using "dedicated" connection

Attribute	Detail
Root Cause	DX provides **private routing** (no public internet traversal) but does **NOT encrypt by default** — traffic on the fiber link is in cleartext

Resolution Options:

Option	Method	When to Use
MACsec	IEEE 802.1AE L2 encryption, AES-256-GCM at line rate	Dedicated connections (1G/10G/100G), requires MACsec-capable routers
IPSec over DX	IPSec tunnel over the private VIF	Hosted connections or when MACsec not available; adds overhead

```
# Best practice for BFSI/regulated industries:
Primary:   Direct Connect 10G + MACsec (L2 encryption at line rate)
Failover:  Site-to-Site VPN over internet (IPSec IKEv2 AES-256)
Both terminate on Transit Gateway
```

Interview Key Point: Direct Connect SLA = 99.9% (single connection), 99.99% (dual connections from different providers/locations).

Issue 2.3 — On-premises cannot resolve AWS internal DNS names

Attribute	Detail
Layer	Hybrid DNS (Route 53 Resolver)
Symptom	On-premises servers can't resolve `rds.cluster.us-east-1.rds.amazonaws.com` or custom `*.aws.internal` names
Root Cause	No Route 53 Resolver Inbound Endpoint configured; on-prem DNS servers are not forwarding AWS queries to the resolver endpoint

Resolution:

```
Architecture:
  On-Prem DNS Server
```

```
    └── Forward rule: *.aws.internal → Route 53 Resolver Inbound Endpoint IP
(10.0.1.10)

  Route 53 Resolver Inbound Endpoint
    └── ENI in private subnet with fixed IP (10.0.1.10)
    └── Receives queries from on-prem, resolves against Private Hosted Zone

  Route 53 Private Hosted Zone: aws.internal
    └── Records: db.aws.internal → RDS endpoint
```

Reverse path (AWS → on-prem): Create an Outbound Endpoint + Resolver Rule forwarding `corp.company.com` to on-premises DNS IP `10.100.0.2`.

---

3. Transit Gateway & Multi-VPC Issues

Issue 3.1 — TGW-attached VPCs cannot communicate (traffic blackholed)

Attribute	Detail
Layer	Transit Gateway Route Tables
Symptom	VPC-A cannot reach VPC-B even though both are attached to the same TGW
Root Cause	(a) TGW route table for VPC-A does not have a route to VPC-B's CIDR; (b) VPC route table does not route VPC-B's CIDR to the TGW attachment; (c) VPCs have overlapping CIDRs

Resolution Checklist:

```
For traffic to flow: VPC-A → TGW → VPC-B

1. VPC-A Route Table:
   10.2.0.0/16 → tgw-<id>    ← Route VPC-B CIDR to TGW

2. TGW Route Table (associated to VPC-A attachment):
   10.2.0.0/16 → VPC-B attachment

3. VPC-B Route Table:
   10.1.0.0/16 → tgw-<id>    ← Return traffic

4. TGW Route Table (associated to VPC-B attachment):
```

```
   10.1.0.0/16 → VPC-A attachment

5. Security groups in both VPCs must allow the cross-VPC traffic
6. No overlapping CIDR ranges between any VPCs
```

TGW Propagation vs. Static Routes: Use BGP propagation for dynamic routing, static routes for explicit traffic steering (e.g., forcing through inspection VPC).

Issue 3.2 — Asymmetric routing breaks stateful firewall inspection (TGW + Palo Alto)

Attribute	Detail
Layer	TGW Appliance Mode / NGFW
Symptom	TCP sessions intermittently drop; Palo Alto logs show asymmetric flows; some connections pass inspection, others don't
Root Cause	TGW without Appliance Mode may send forward and return packets of the same TCP flow to **different NGFW instances** in different AZs, breaking stateful inspection

Resolution:

```
# Enable Appliance Mode on the Inspection VPC TGW attachment
aws ec2 modify-transit-gateway-vpc-attachment \
  --transit-gateway-attachment-id tgw-attach-xxx \
  --options ApplianceModeSupport=enable

# Verify
aws ec2 describe-transit-gateway-vpc-attachments \
  --transit-gateway-attachment-ids tgw-attach-xxx \
  --query 'TransitGatewayVpcAttachments[].Options.ApplianceModeSupport'
```

Why it works: Appliance Mode pins both directions of a flow to the same AZ within the TGW, ensuring the same NGFW instance sees both the request and response — required for stateful inspection.

Issue 3.3 — CIDR overlap prevents TGW or VPC peering

Attribute	Detail
Layer	VPC Design / Planning
Symptom	Cannot add VPC to TGW; peering request fails with "overlapping CIDR block" error
Root Cause	Two VPCs share the same IP range (e.g., both use 10.0.0.0/16)

Prevention — CIDR Allocation Plan:

```
Organization CIDR: 10.0.0.0/8
  ├── Shared Services VPC:  10.0.0.0/16
  ├── Dev VPC:              10.1.0.0/16
  ├── Staging VPC:          10.2.0.0/16
  ├── Production VPC:       10.3.0.0/16
  ├── Security VPC:         10.4.0.0/16
  ├── On-Premises:          10.100.0.0/16
  └── Reserved for growth:  10.5.0.0/16 – 10.99.0.0/16
```

If overlap exists and migration is needed: Create new VPC with non-overlapping CIDR, migrate workloads gradually. Use AWS Migration Hub for tracking.

---

4. Compute Issues (EC2 / Auto Scaling)

Issue 4.1 — Auto Scaling does not scale out during traffic spike

Attribute	Detail
Layer	Auto Scaling / CloudWatch
Symptom	CPU goes to 100%, ALB latency spikes, but no new instances are launched

Attribute	Detail
Root Cause	(a) Scaling policy is not attached; (b) Cooldown period too long (instances launched but scaling suppressed); (c) Max capacity already reached; (d) Service quota hit (vCPU limit); (e) AMI launch failure

Resolution:

```
# Check ASG activity history (shows what happened and why)
aws autoscaling describe-scaling-activities \
  --auto-scaling-group-name my-asg --max-items 20

# Common findings:
# "Launch failed: insufficient capacity" → Request Spot or different
instance type/AZ
# "Launch failed: vpc subnet exhausted" → Subnet ran out of IPs
# "InCooldown" → Previous scale activity still in cooldown; reduce cooldown
period

# Check current/max capacity
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names my-
asg \
  --query 'AutoScalingGroups[].
{Min:MinSize,Max:MaxSize,Desired:DesiredCapacity}'
```

Best Practices:

- Use **target tracking** scaling policies (simpler, AWS manages the math)
- Set **warmup period** on individual instances, not cooldown on the group
- Use **mixed instances policy** with multiple instance types and AZs to avoid single-AZ capacity exhaustion

---

Issue 4.2 — EC2 instance becomes unreachable (no SSH / RDP)

Attribute	Detail
Layer	EC2 Connectivity
Symptom	SSH/RDP connection timeout to EC2 instance
Root Cause (multiple)	(a) Security group doesn't allow SSH (port 22) from your IP; (b) Instance doesn't have a public IP (private subnet, no bastion); (c)

Attribute	Detail
	Instance in failed state; (d) OS-level firewall blocking

Resolution Approach:

```
1. Check Security Group: Inbound SSH/RDP from your source IP?

2. Check Instance State:
   aws ec2 describe-instance-status --instance-ids i-xxx
   → "reachability: passed" = OS is running
   → "impaired" = instance has a problem

3. Instance in private subnet:
   → Use Systems Manager Session Manager (no SSH needed, no bastion needed)
   aws ssm start-session --target i-xxx
   → Prerequisites: SSM Agent running, SSM VPC endpoints or internet access

4. Network unreachable:
   → Use EC2 Serial Console (for emergency access even with no networking)

5. OS issue (kernel panic):
   → Detach root EBS, attach to rescue instance, fix, re-attach
```

Best Practice: Never open port 22 to `0.0.0.0/0`. Use SSM Session Manager instead of SSH — removes the need for a bastion host entirely.

Issue 4.3 — Spot Instance interrupted unexpectedly, causing service disruption

Attribute	Detail
Layer	Spot Instances / Resilience
Symptom	Application crashed; 2-minute interruption notice was not handled gracefully
Root Cause	Application had no handler for the Spot interruption notice (IMDS metadata endpoint); single instance without diversification

Resolution:

```
# Handle Spot interruption notice via IMDS endpoint
import requests
import time

def check_spot_interruption():
    try:
        # Check for interruption notice (available 2 min before termination)
        r = requests.get(
            "http://169.254.169.254/latest/meta-data/spot/interruption-
action",
            timeout=1
        )
        if r.status_code == 200:
            # "terminate" or "stop" or "hibernate"
            print(f"Spot interruption notice: {r.text}")
            # Graceful shutdown: drain connections, checkpoint work
            graceful_shutdown()
    except requests.exceptions.ConnectionError:
        pass  # No interruption
```

Architecture Fix:

- Never run production on a single Spot instance
- Diversify across multiple instance types (ASG mixed instances policy)
- Use On-Demand for minimum baseline capacity + Spot for burst
- Store state externally (S3, DynamoDB, ElastiCache) — not on local disk

---

5. ECS & Fargate Issues

Issue 5.1 — ECS task fails to start (STOPPED with exit code 1)

Attribute	Detail
Layer	ECS / Container Runtime
Symptom	Tasks continuously stop with `Essential container exited`; service can't maintain desired count
Root Cause	(a) Container application crashes on startup (check application logs); (b) Missing environment variables or secrets; (c) IAM task role missing permissions

Resolution:

```
# 1. Find the stopped task and get reason
aws ecs list-tasks --cluster my-cluster --desired-status STOPPED
aws ecs describe-tasks --cluster my-cluster --tasks <task-arn> \
  --query 'tasks[].containers[].{name:name,exitCode:exitCode,reason:reason}'

# 2. Check CloudWatch Logs for the container
aws logs get-log-events \
  --log-group-name /ecs/my-service \
  --log-stream-name ecs/my-container/<task-id>

# 3. Common exit codes:
# Exit 1: Application error (check logs)
# Exit 137: OOM killed (increase memory in task definition)
# Exit 139: Segfault
# Exit 143: SIGTERM not handled (graceful shutdown issue)
```

Issue 5.2 — Fargate task cannot pull image from ECR (ImagePullBackOff)

Attribute	Detail
Layer	Fargate / ECR / Networking
Symptom	Task stops with `CannotPullContainerError: pull access denied`
Root Cause	(a) Task execution role missing `ecr:GetAuthorizationToken`, `ecr:GetDownloadUrlForLayer` permissions; (b) Fargate in private subnet without VPC Endpoints for ECR; (c) ECR image URI wrong

Resolution:

```
// Required permissions on ECS Task Execution Role:
{
  "Effect": "Allow",
  "Action": [
    "ecr:GetAuthorizationToken",
    "ecr:BatchCheckLayerAvailability",
    "ecr:GetDownloadUrlForLayer",
    "ecr:BatchGetImage"
  ],
```

```
    "Resource": "*"
  }
```

For private subnets (no internet):

```
Required VPC Endpoints for Fargate ECR access:
  ✦ com.amazonaws.region.ecr.api      (Interface)
  ✦ com.amazonaws.region.ecr.dkr      (Interface)
  ✦ com.amazonaws.region.s3           (Gateway - free)
  ✦ com.amazonaws.region.logs         (Interface - for CloudWatch)
```

---

6. EKS / Kubernetes Issues

Issue 6.1 — Pod stuck in CrashLoopBackOff

Attribute	Detail
Layer	Kubernetes / Application
Symptom	Pod restarts repeatedly; `RESTARTS` count keeps increasing

Debugging Process:

```
# Step 1: See events and exit code
kubectl describe pod <pod-name> -n <namespace>
# Look for: "Exit Code", "Last State", "Reason"

# Step 2: Read logs from the crashed container
kubectl logs <pod-name> -n <namespace> --previous

# Step 3: Common exit codes and fixes:
```

Exit Code	Meaning	Fix
137	OOMKilled — JVM/process exceeded memory limit	Increase `resources.limits.memory`; tune JVM with `-XX:MaxRAMPercentage=75.0`

Exit Code	Meaning	Fix
1	Application error	Check logs for stack trace; missing env vars; DB connection failure
143	SIGTERM — graceful shutdown timeout	Implement signal handler; increase `terminationGracePeriodSeconds`
Readiness failure	App starts but `/health` returns DOWN	Increase `initialDelaySeconds`; check DB connectivity from pod

---

Issue 6.2 — Pod stuck in Pending state

Attribute	Detail
Layer	Kubernetes Scheduling
Symptom	Pod shows `Pending` for minutes; never scheduled

Resolution Decision Tree:

```
kubectl describe pod <pod-name> | grep -A 5 "Events:"

# "0/3 nodes are available: 3 Insufficient cpu/memory"
→ Nodes are full; scale node group up OR reduce pod requests

# "0/3 nodes are available: 3 node(s) had untolerated taint"
→ Pod needs toleration for the taint on those nodes
→ Fix: Add tolerations to pod spec

# "0/3 nodes are available: 3 node(s) didn't match Pod's node affinity"
→ Node labels don't match nodeAffinity requirements
→ Fix: Label the correct nodes OR fix the affinity rules

# "PersistentVolumeClaim not bound"
→ StorageClass provisioner cannot create volume
→ Check EBS CSI driver, IAM permissions, AZ constraints
```

Issue 6.3 — Pods cannot communicate across nodes (intermittent 502s)

Attribute	Detail
Layer	Kubernetes Networking / VPC CNI
Symptom	Requests to a service sometimes succeed, sometimes fail (depends on which pod handles it)
Root Cause	(a) NetworkPolicy default-deny blocking certain paths; (b) VPC CNI version issue; (c) kube-proxy iptables stale rules; (d) Pod not in Endpoints list (readiness probe failing)

Resolution:

```
# 1. Check if endpoints are populated
kubectl get endpoints <service-name> -n <namespace>
# Empty? → Label mismatch or pods not ready

# 2. Check NetworkPolicies
kubectl get networkpolicies -n <namespace>
# Look for default-deny and verify allow rules cover your traffic path

# 3. Check VPC CNI status
kubectl get pods -n kube-system -l k8s-app=aws-node
kubectl logs -n kube-system -l k8s-app=aws-node | grep -i error

# 4. Debug with netshoot
kubectl run debug --image=nicolaka/netshoot -it --rm -- /bin/bash
# Inside: curl, dig, tcpdump, iperf for network testing
```

Issue 6.4 — IRSA not working (pod cannot access AWS services)

Attribute	Detail
Layer	IRSA / IAM / EKS
Symptom	Pod gets `AccessDenied` calling AWS APIs despite IRSA setup
Root Cause	(a) ServiceAccount annotation missing or wrong ARN; (b) IAM trust policy OIDC condition mismatch; (c) Pod doesn't use the annotated ServiceAccount

Resolution:

```
# 1. Verify ServiceAccount annotation
kubectl get serviceaccount my-sa -n my-namespace -o yaml
# Must have: eks.amazonaws.com/role-arn: arn:aws:iam::ACCOUNT:role/my-role

# 2. Verify IAM role trust policy
aws iam get-role --role-name my-role --query 'Role.AssumeRolePolicyDocument'
# Must match OIDC provider URL and namespace/serviceaccount

# 3. Verify pod uses the ServiceAccount
kubectl get pod my-pod -o yaml | grep serviceAccountName
# Must be "my-sa"

# 4. Verify AWS_ROLE_ARN env var is injected
kubectl exec my-pod -- env | grep AWS_ROLE_ARN
# Should be set by the EKS Pod Identity Webhook
```

IAM Trust Policy Example (must be exact):

```
{
  "Effect": "Allow",
  "Principal": {
    "Federated": "arn:aws:iam::ACCOUNT:oidc-
provider/oidc.eks.REGION.amazonaws.com/id/CLUSTER_ID"
  },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "oidc.eks.REGION.amazonaws.com/id/CLUSTER_ID:sub":
"system:serviceaccount:NAMESPACE:SA_NAME",
      "oidc.eks.REGION.amazonaws.com/id/CLUSTER_ID:aud": "sts.amazonaws.com"
    }
  }
}
```

Issue 6.5 — HPA not scaling (pods not scaling despite high CPU)

Attribute	Detail
Layer	Kubernetes HPA / Metrics Server
Symptom	CPU at 90% but HPA shows `unknown` for current metrics; pods not scaling

Attribute	Detail
Root Cause	Metrics Server not installed; pods missing `resources.requests.cpu` (HPA calculates as percentage of request, not node CPU); Custom metric adapter not configured

Resolution:

```
# Check HPA status
kubectl describe hpa my-deployment -n my-namespace
# "unable to get metrics for resource cpu" → Metrics Server missing

# Install Metrics Server
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
server/releases/latest/download/components.yaml

# Verify pod has resource requests (REQUIRED for HPA)
kubectl get deployment my-deployment -o yaml | grep -A 5 resources
# Must have: requests.cpu defined

# Check current metrics
kubectl top pods -n my-namespace
```

Issue 6.6 — Karpenter not provisioning nodes (pods stuck Pending)

Attribute	Detail
Layer	Karpenter / EKS Node Provisioning
Symptom	Pods are Pending but Karpenter doesn't provision nodes
Root Cause	(a) No matching NodePool (provisioner) for pod's requirements; (b) EC2 instance launch failure (quota, SCP); (c) IRSA for Karpenter node role missing permissions

Resolution:

```
# Check Karpenter logs
kubectl logs -n kube-system -l app.kubernetes.io/instance=karpenter -f

# Common log messages:
# "no instance type found" → NodePool instance type requirements too
```

```
restrictive
# "failed to launch instance" → IAM permissions or EC2 quota issue
# "disruption disabled" → Drift/Consolidation disabled

# Check for Spot capacity issues
kubectl logs -n kube-system -l app.kubernetes.io/instance=karpenter | grep
"insufficient capacity"
# Fix: Add more instance type options to NodePool
```

---

7. EKS Upgrade Issues (1.29 → 1.31)

Issue 7.1 — Control plane upgrade stuck or failed

Error	Root Cause	Fix
`Update is in FAILED status`	SCP blocking `eks:UpdateClusterVersion`; or subnet IP exhaustion	Check CloudTrail for `AccessDenied`; verify subnets have ≥5 free IPs per subnet
`kubectl` times out after upgrade	Private cluster — API server endpoint changed	Run `aws eks update-kubeconfig --name cluster` to refresh kubeconfig
`Unable to connect to server` post-upgrade	Missing VPC endpoint for EKS API	Create `com.amazonaws.region.eks` VPC Interface Endpoint
Upgrade blocked on API deprecation	Resources using removed API versions	Run `kubent` pre-upgrade; update deprecated `apiVersion` fields

Pre-upgrade API deprecation check:

```
# Install kubent (Kube-No-Trouble)
sh -c "$(curl -sSL https://git.io/install-kubent)"
kubent
# Output shows deprecated APIs still in use — fix BEFORE upgrading

# Manual fix example (FlowSchema v1beta2 → v1):
kubectl get flowschema system-leader -o yaml | \
  sed 's/apiVersion: flowcontrol.apiserver.k8s.io\/v1beta2/apiVersion:
flowcontrol.apiserver.k8s.io\/v1/' | \
  kubectl apply -f -
```

Issue 7.2 — Node group upgrade breaks pod connectivity to Aurora

Attribute	Detail
Layer	EKS Upgrade / Aurora / Security Groups
Symptom	After node group upgrade, pods get `Connection refused` to Aurora on port 5432
Root Cause	New node group has a **new security group ID**. Aurora's inbound rule references the old node group SG ID. New nodes are not in Aurora's allow-list

Resolution:

```
# Get new node group SG ID
aws eks describe-nodegroup --cluster-name my-cluster --nodegroup-name new-ng
\
  --query 'nodegroup.resources.remoteAccessSecurityGroup'

# Add the new node SG to Aurora's security group
aws ec2 authorize-security-group-ingress \
  --group-id sg-aurora-xxx \
  --protocol tcp \
  --port 5432 \
  --source-group sg-new-node-xxx

# Verify
aws rds describe-db-cluster-snapshot ... # or check via console
```

Prevention: Reference **node group role** in Aurora SG instead of the SG ID, or use a static "EKS Node" SG that persists across node group rotations.

Issue 7.3 — Aurora connection storm after node drain

Attribute	Detail
Layer	EKS Upgrade / Database Connections
Symptom	All pods restart simultaneously during node drain → Aurora gets hundreds of new connections at once → `too many connections` error → cascade failure
Root Cause	No connection pooling; all pods reconnect simultaneously when rescheduled on new nodes

Resolution:

Option A — RDS Proxy (recommended for production):

```
resource "aws_db_proxy" "aurora_proxy" {
  name                   = "aurora-proxy"
  engine_family          = "POSTGRESQL"
  vpc_subnet_ids         = var.private_subnet_ids
  vpc_security_group_ids = [aws_security_group.rds_proxy.id]

  auth {
    auth_scheme = "SECRETS"
    secret_arn  = aws_secretsmanager_secret.db_creds.arn
    iam_auth    = "REQUIRED"
  }
}
# RDS Proxy absorbs the connection storm; maintains a pool of persistent
connections to Aurora
```

Option B — Staggered pod startup:

```
spec:
  strategy:
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0   # Ensures only 1 pod at a time changes
```

```
  template:
    spec:
      containers:
      - name: app
        readinessProbe:
          initialDelaySeconds: 30  # Give time for DB connection pool to
initialize
```

Issue 7.4 — Add-on upgrade causes CoreDNS disruption (DNS failures cluster-wide)

Attribute	Detail
Layer	EKS Add-ons / CoreDNS
Symptom	Immediately after CoreDNS add-on upgrade, pods get DNS resolution failures; services can't find each other
Root Cause	Rolling restart of CoreDNS pods during upgrade; if only 1 replica or no PDB, both pods may restart simultaneously

Resolution:

```
# Ensure CoreDNS has at least 2 replicas before upgrade
kubectl get deployment coredns -n kube-system
kubectl scale deployment coredns -n kube-system --replicas=3

# Add PDB to CoreDNS before upgrading
cat <<EOF | kubectl apply -f -
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: coredns-pdb
  namespace: kube-system
spec:
  minAvailable: 1
  selector:
    matchLabels:
      k8s-app: kube-dns
EOF

# Monitor during upgrade
kubectl rollout status deployment/coredns -n kube-system --timeout=5m
```

Issue 7.5 — Pods stuck in Terminating during node drain

Attribute	Detail
Layer	EKS Upgrade / PodDisruptionBudgets
Symptom	`kubectl drain` hangs; pods stuck in `Terminating` for 10+ minutes
Root Cause	(a) PDB too restrictive (`minAvailable = replicas` — impossible to drain); (b) Pod has a finalizer that never completes; (c) Webhook blocking pod deletion

Resolution:

```
# Check PDB
kubectl get pdb --all-namespaces
# If "ALLOWED DISRUPTIONS = 0" → PDB is too restrictive
# Fix: Scale deployment up first, THEN drain
kubectl scale deployment my-app -n my-ns --replicas=3
# Now PDB allows 1 disruption

# Force delete stuck pod (last resort — only if pod is truly stuck)
kubectl delete pod <pod-name> --grace-period=0 --force

# Check for finalizers preventing deletion
kubectl get pod <pod-name> -o yaml | grep -A 5 finalizers
# If present: kubectl patch pod <pod-name> -p '{"metadata":{"finalizers":
[]}}'
```

---

8. Lambda & Serverless Issues

Issue 8.1 — Lambda timeout errors

Attribute	Detail
Layer	Lambda / Serverless
Symptom	Lambda function logs `Task timed out after X seconds`

Attribute	Detail
Root Cause	(a) Function timeout set too low (default 3s); (b) Function in VPC — cold start takes longer to attach ENI; (c) Downstream service (RDS, API) taking too long

Resolution:

```
# Increase timeout (max 15 minutes)
aws lambda update-function-configuration \
  --function-name my-function \
  --timeout 30

# For VPC Lambda: ENI attachment adds ~500ms-1s to cold start
# Mitigation: Keep Lambda "warm" with provisioned concurrency
aws lambda put-provisioned-concurrency-config \
  --function-name my-function \
  --qualifier production \
  --provisioned-concurrent-executions 5

# For downstream timeout: implement exponential backoff with jitter
import time, random
def retry_with_backoff(func, max_attempts=3):
    for attempt in range(max_attempts):
        try:
            return func()
        except Exception as e:
            if attempt == max_attempts - 1:
                raise
            wait = (2 ** attempt) + random.uniform(0, 1)
            time.sleep(wait)
```

Issue 8.2 — Lambda in VPC cannot connect to internet or RDS

Attribute	Detail
Layer	Lambda / VPC Networking
Symptom	Lambda times out calling external APIs; or can't reach RDS

Attribute	Detail
Root Cause	Lambda in VPC uses VPC networking. Unlike EC2, Lambda does NOT use the ENI's subnet NAT automatically — **must have NAT Gateway in a public subnet** and Lambda must be in a **private subnet** with route to NAT

Resolution Architecture:

```
Lambda ENI (private subnet) → Route Table → NAT Gateway (public subnet) →
IGW → Internet
                                          → RDS SG (allow Lambda SG on port
5432)
```

Common Mistake: Placing Lambda in a **public subnet** with IGW does NOT give it internet access — Lambda ENIs don't get public IPs. Lambda must be in a **private subnet** pointing to NAT.

Issue 8.3 — Lambda throttling (concurrent execution limit)

Attribute	Detail
Layer	Lambda Concurrency
Symptom	Lambda invocations return `TooManyRequestsException` (status 429)
Root Cause	Account-level concurrency limit (default 1000 concurrent executions) reached; OR function-level reserved concurrency too low

Resolution:

```
# Check current concurrency usage
aws lambda get-account-settings

# Request limit increase
aws service-quotas request-service-quota-increase \
  --service-code lambda \
  --quota-code L-B99A9384 \
  --desired-value 3000
```

```
# Set reserved concurrency to protect critical functions
aws lambda put-function-concurrency \
  --function-name critical-payment-function \
  --reserved-concurrent-executions 200

# Implement SQS queue as buffer for bursty workloads
# Lambda polls SQS → processes at own pace → no throttling from burst
```

---

9. IAM & Access Control Issues

Issue 9.1 — `AccessDenied` despite IAM policy appearing correct

Attribute	Detail
Layer	IAM
Symptom	API call fails with `AccessDenied` even though the IAM policy has the action allowed
Root Cause	IAM uses **explicit deny wins** logic. Something is denying: (a) SCP at organization level; (b) Permission boundary; (c) Resource policy denying the principal; (d) VPC endpoint policy blocking the call

Systematic debugging:

```
# Use IAM Policy Simulator (most effective)
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::ACCOUNT:role/my-role \
  --action-names s3:GetObject \
  --resource-arns arn:aws:s3:::my-bucket/object.txt

# Check if SCP is blocking
aws organizations list-policies-for-target \
  --target-id ACCOUNT_ID \
  --filter SERVICE_CONTROL_POLICY

# Check CloudTrail for the exact denied call
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=GetObject \
  --start-time $(date -v-1H +%Y-%m-%dT%H:%M:%S)
```

IAM Evaluation Order:

```
1. Explicit DENY (anywhere) → DENIED immediately
2. Organizational SCP → must ALLOW
3. Resource policy → checked
4. IAM identity policy → must ALLOW
5. Permission boundary → must ALLOW (if set)
6. Session policy → must ALLOW (if assumed role with conditions)
All must allow → ACCESS GRANTED
```

Issue 9.2 — Stored access keys leaked / compromised

Attribute	Detail
Layer	IAM / Security
Symptom	AWS Support notifies of compromised key; GuardDuty alert `UnauthorizedAccess:IAMUser/AnomalousBehavior`
Root Cause	Long-lived access keys hardcoded in code, committed to Git, or stored in CI/CD secrets

Immediate Response:

```
# 1. IMMEDIATELY deactivate/delete the key
aws iam delete-access-key \
  --user-name compromised-user \
  --access-key-id AKIAIOSFODNN7EXAMPLE

# 2. Check what was done with the key (last 90 days)
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=Username,AttributeValue=compromised-user
\
  --max-results 50

# 3. Check for unauthorized resources (cryptomining is common)
aws ec2 describe-instances --filter Name=instance-state-name,Values=running
aws lambda list-functions
aws s3api list-buckets

# 4. Rotate any downstream secrets the key may have accessed
# 5. File a security incident report
```

Prevention (the real fix):

- **Eliminate all access keys.** Use OIDC federation for CI/CD (GitHub Actions, GitLab CI)
- EC2 → Instance Profile; ECS → Task Role; EKS → IRSA; Lambda → Execution Role
- Enforce with SCP: `Deny iam:CreateAccessKey` for all non-service accounts
- Enable Gitleaks in pre-commit hooks to catch secrets before they're committed

---

Issue 9.3 — Cross-account role assumption failing

Attribute	Detail
Layer	IAM / Cross-Account
Symptom	`sts:AssumeRole` fails with `AccessDenied`
Root Cause	Both conditions must be true: (1) Source account's IAM principal must have `sts:AssumeRole` permission for the target role; (2) Target role's trust policy must list the source principal

Resolution:

```
// Source account — IAM policy on the user/role:
{
  "Effect": "Allow",
  "Action": "sts:AssumeRole",
  "Resource": "arn:aws:iam::TARGET_ACCOUNT:role/cross-account-role"
}

// Target account — Trust policy on the role (cross-account-role):
{
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::SOURCE_ACCOUNT:role/deployer-role"
  },
  "Action": "sts:AssumeRole"
}
```

---

10. Database Issues (RDS, Aurora, DynamoDB)

Issue 10.1 — RDS Multi-AZ failover takes too long / application not reconnecting

Attribute	Detail
Layer	RDS / High Availability
Symptom	After RDS Multi-AZ failover (AZ outage), application is still trying to connect to old instance IP
Root Cause	Application is caching the DNS resolved IP of the RDS endpoint; RDS failover changes which instance the DNS points to, but app has stale cache

Resolution:

1. Ensure application uses **RDS endpoint DNS name**, not cached IP
2. Set DNS TTL handling in connection string (JDBC: `connectTimeout`, `socketTimeout`)
3. Implement **exponential backoff with retry** in connection pool:

```
// HikariCP (Java) — handles reconnection automatically
HikariConfig config = new HikariConfig();
config.setConnectionTimeout(30000);
config.setKeepaliveTime(60000);
config.setMaxLifetime(1800000);
config.setConnectionInitSql("SELECT 1");
// Validates connection on borrow — replaces stale connections
```

4. Use **RDS Proxy** — maintains connection pool; handles failover transparently, app-side connection stays valid

---

Issue 10.2 — Aurora writer instance overloaded; read queries saturating it

Attribute	Detail
Layer	Aurora / Read Scaling
Symptom	Aurora writer CPU > 80%; slow queries; `max_connections` errors
Root Cause	All queries (reads + writes) going to the writer; read replicas exist but application not using them

Resolution:

```
# Use cluster reader endpoint for read queries
WRITER_ENDPOINT = "cluster.cluster-xxxx.rds.amazonaws.com"    # Read/Write
READER_ENDPOINT = "cluster.cluster-ro-xxxx.rds.amazonaws.com"  # Read Only

# Route read vs write at application layer
def get_connection(readonly=False):
    host = READER_ENDPOINT if readonly else WRITER_ENDPOINT
    return create_engine(f"postgresql://user:pass@{host}:5432/db")

# OR use ProxySQL / PgBouncer for transparent read/write splitting
```

Scale further: Add up to 15 Aurora read replicas. Use Aurora Auto Scaling to add/remove replicas based on `ReaderAverageDatabaseConnections` metric.

Issue 10.3 — DynamoDB hot partition causing throttling

Attribute	Detail
Layer	DynamoDB / NoSQL
Symptom	`ProvisionedThroughputExceededException`; certain items accessed far more than others
Root Cause	Partition key is not evenly distributed (e.g., using date as partition key concentrates today's traffic on one partition)

Resolution:

```
Option 1: Shard the partition key
  Instead of partition_key = "2026-06-16"
```

```
  Use partition_key = "2026-06-16-{random_suffix_0_9}"
  Distribute across 10 partitions

Option 2: DAX (DynamoDB Accelerator) for read-heavy hotspots
  - In-memory cache in front of DynamoDB
  - Sub-millisecond reads
  - Transparent to application (same API)

Option 3: Switch to On-Demand capacity
  - No pre-provisioning; DynamoDB auto-scales
  - Costs more per request but handles any burst
  - Ideal when traffic is unpredictable

Option 4: Write-behind cache
  - Write to ElastiCache first → async persist to DynamoDB
  - Reduces direct DynamoDB write pressure
```

Issue 10.4 — RDS out of storage; disk full → instance unavailable

Attribute	Detail
Layer	RDS Storage
Symptom	RDS instance becomes read-only or unavailable; CloudWatch `FreeStorageSpace` drops to 0
Root Cause	Uncontrolled data growth; large binary columns; WAL/transaction log accumulation

Resolution:

```
# Immediate: Enable Storage Auto Scaling (if not already)
aws rds modify-db-instance \
  --db-instance-identifier my-db \
  --max-allocated-storage 1000 \  # in GB
  --apply-immediately

# Or manually increase storage
aws rds modify-db-instance \
  --db-instance-identifier my-db \
  --allocated-storage 500 \
  --apply-immediately

# Identify large tables
# Login to DB: SELECT schemaname, tablename,
pg_size_pretty(pg_total_relation_size(schemaname || '.' || tablename))
```

```
# FROM pg_tables ORDER BY pg_total_relation_size(schemaname || '.' ||
tablename) DESC LIMIT 10;
```

Prevention:

- Enable **RDS Storage Autoscaling** with a maximum size limit
- CloudWatch alarm: `FreeStorageSpace < 20%` → page on-call
- Implement data lifecycle: archive old records to S3, delete unnecessary data

---

11. Storage Issues (S3, EBS, EFS)

Issue 11.1 — S3 unexpected public access / data exposure

Attribute	Detail
Layer	S3 / Security
Symptom	Sensitive data accessible publicly; Security Hub finding for public S3 bucket
Root Cause	Bucket-level or object-level ACL set to public; or bucket policy with `"Principal": "*"` without sufficient conditions

Immediate Remediation:

```
# Block all public access (account-level guardrail)
aws s3control put-public-access-block \
  --account-id ACCOUNT_ID \
  --public-access-block-configuration \

BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPu
blicBuckets=true

# Per-bucket
aws s3api put-public-access-block \
  --bucket my-bucket \
  --public-access-block-configuration \

"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictP
ublicBuckets=true"

# Audit all buckets for public access
```

```
aws s3api list-buckets --query 'Buckets[].Name' --output text | \
  xargs -I {} aws s3api get-bucket-acl --bucket {}
```

Prevention via SCP:

```
{
  "Effect": "Deny",
  "Action": ["s3:PutBucketAcl", "s3:PutObjectAcl"],
  "Resource": "*",
  "Condition": {
    "StringEquals": {
      "s3:x-amz-acl": ["public-read", "public-read-write", "authenticated-
read"]
    }
  }
}
```

Issue 11.2 — EBS volume reaching IOPS limit causing I/O wait

Attribute	Detail
Layer	EBS / Performance
Symptom	High `%iowait` in top/iostat; slow application response; `BurstBalance` metric at 0%
Root Cause	gp2 volume using burst I/O credits (which deplete for volumes < 1TB); or volume type doesn't have sufficient baseline IOPS

Resolution:

```
# Upgrade gp2 to gp3 (immediately better, no restart needed)
aws ec2 modify-volume \
  --volume-id vol-xxx \
  --volume-type gp3 \
  --iops 3000 \           # Baseline IOPS (configurable, not credit-based)
  --throughput 125        # MB/s

# gp3 vs gp2:
# gp3: 3000 IOPS baseline always (no burst credits); 20% cheaper; up to
16,000 IOPS
# gp2: 3 IOPS/GB; burst up to 3000 for small volumes (depletes)
```

```
# Monitor:
# CloudWatch: VolumeQueueLength > 1 sustained → IOPS bottleneck
# BurstBalance → 0 → gp2 volume exhausted burst credits
```

---

12. DevOps — CI/CD Pipeline Issues

Issue 12.1 — GitHub Actions OIDC authentication fails for AWS

Attribute	Detail
Layer	CI/CD / IAM OIDC
Symptom	GitHub Actions workflow fails at `configure-aws-credentials` step with `AccessDenied` or `InvalidIdentityToken`
Root Cause	(a) Trust policy condition uses wrong claim format; (b) OIDC provider thumbprint not updated; (c) Permission `id-token: write` missing from workflow

Resolution:

```
# Workflow MUST have this permission
permissions:
  id-token: write   # REQUIRED for OIDC
  contents: read

steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::ACCOUNT:role/github-actions-role
      aws-region: us-east-1
      # role-session-name optional but helps with CloudTrail attribution
      role-session-name: github-${{ github.run_id }}
```

Trust Policy (exact syntax required):

```
{
  "StringLike": {
    "token.actions.githubusercontent.com:sub":
"repo:ORG/REPO:ref:refs/heads/main"
```

```
    },
    "StringEquals": {
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
    }
  }
```

Tighten for security: Use `StringEquals` with exact branch for production deployments; use `StringLike` only for pattern matching multiple branches.

Issue 12.2 — Docker build fails in CI (no space left / cache issues)

Attribute	Detail
Layer	CI/CD / Docker
Symptom	CI fails with `no space left on device`; builds getting slower over time
Root Cause	Dangling Docker layers and images accumulating on self-hosted runners

Resolution:

```
# In GitHub Actions or GitLab CI — prune Docker before/after builds
- name: Prune Docker cache
  run: docker system prune -f --volumes

# Use multi-stage builds to reduce layer size:
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline   # Cache dependencies in separate layer
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:17-jre-jammy  # Minimal runtime image
COPY --from=builder /app/target/*.jar app.jar
USER nonroot
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Issue 12.3 — Deployment pipeline bypasses required approvals

Attribute	Detail
Layer	CI/CD / Governance
Symptom	Changes to production were deployed without change advisory board (CAB) review
Root Cause	No environment protection rules; direct push to main allowed; pipeline has no approval gate for production

Resolution (GitHub Actions Environment Protection):

```
jobs:
  deploy-prod:
    environment:
      name: production          # Requires approval from "production"
environment reviewers
      url: https://app.prod.com
    runs-on: ubuntu-latest
    needs: deploy-staging       # Must pass staging first
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to Production
        run: ./deploy.sh prod
```

Enable in GitHub Settings: `Settings > Environments > production > Required reviewers (list approvers) + Required wait timer`

---

13. Terraform / IaC Issues

Issue 13.1 — `terraform apply` proposes destroying a production database

Attribute	Detail
Layer	Terraform / IaC

Attribute	Detail
Symptom	`plan` output shows `# aws_rds_cluster.main will be destroyed`
Root Cause	(a) Resource renamed in Terraform code (Terraform sees: destroy old, create new); (b) Force-replacement argument changed (e.g., `db_name`, `engine`); (c) State file mismatch

Immediate Action: Cancel the apply (Ctrl+C)! Never blindly apply.

Diagnosis and Fix:

```
# Scenario A: Resource was renamed in code
# Old: resource "aws_rds_cluster" "aurora" {}
# New: resource "aws_rds_cluster" "main" {}
# Fix: Move state, don't destroy
terraform state mv aws_rds_cluster.aurora aws_rds_cluster.main
terraform plan  # Should now show: No changes

# Scenario B: Force-replacement argument changed
# Check: terraform plan -detailed-exitcode
# If "forces replacement" — revert the parameter in code

# Prevention: Use lifecycle block
resource "aws_rds_cluster" "main" {
  lifecycle {
    prevent_destroy = true  # Terraform will error instead of destroy
    ignore_changes  = [master_password]  # Don't track rotation
  }
}
```

Issue 13.2 — Terraform state lock not releasing (DynamoDB lock stuck)

Attribute	Detail
Layer	Terraform / State
Symptom	`Error: Error locking state: Error acquiring the state lock` — another process holds it
Root Cause	Previous `terraform apply` was interrupted (Ctrl+C, CI failure, network drop) without releasing the DynamoDB lock

Resolution:

```
# View the lock (find LockID)
aws dynamodb get-item \
  --table-name terraform-state-lock \
  --key '{"LockID": {"S": "my-project/production/terraform.tfstate"}}'

# Force unlock (use LockID from the error message or DynamoDB item)
terraform force-unlock <LockID>

# Verify: should now be able to run terraform plan/apply
```

Prevention:

- Always use DynamoDB state locking
- Set CI/CD timeout to automatically cancel abandoned jobs
- Use Terraform Cloud/Atlantis for managed state with automatic lock cleanup

---

Issue 13.3 — Terraform state file corrupted or accidentally deleted

Attribute	Detail
Layer	Terraform / State
Symptom	`terraform plan` shows all resources will be recreated; `terraform show` shows empty state
Root Cause	State bucket versioning disabled; manual edit of state file; S3 bucket deleted

Resolution:

```
# Option A: Restore from S3 versioning (if enabled — it should always be)
aws s3api list-object-versions \
  --bucket terraform-state-bucket \
  --prefix prod/terraform.tfstate

aws s3api get-object \
  --bucket terraform-state-bucket \
  --key prod/terraform.tfstate \
  --version-id <good-version-id> \
  terraform.tfstate.restored
```

```
aws s3 cp terraform.tfstate.restored s3://terraform-state-
bucket/prod/terraform.tfstate

# Option B: Rebuild state by importing existing resources
terraform import aws_vpc.main vpc-0abc123def
terraform import aws_eks_cluster.main my-cluster
# Import every resource; then run terraform plan to verify no changes
```

Required S3 Backend Configuration:

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state"
    key            = "prod/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true           # AES-256 / KMS
    dynamodb_table = "terraform-locks"
    versioning     = true           # ALWAYS enable versioning
  }
}
```

Issue 13.4 — Terraform module version drift across environments

Attribute	Detail
Layer	Terraform / IaC Governance
Symptom	Dev uses module `v1.5.0`, production uses `v1.2.0` — diverging infrastructure behavior
Root Cause	No pinned module version in environment configurations; teams upgrade dev but forget prod

Resolution — Module version pinning:

```
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "20.8.5"   # ALWAYS pin to exact version; never use ">=" without
upper bound
}

# For internal modules via Git:
```

```
module "vpc" {
  source = "git::https://github.com/myorg/terraform-modules.git//vpc?
ref=v2.1.0"
  # ref = Git tag or commit SHA; never use main/master branch
}
```

---

14. DevSecOps — Security Scanning Issues

Issue 14.1 — Secrets committed to Git repository

Attribute	Detail
Layer	DevSecOps / Secrets Management
Symptom	Gitleaks or Trufflehog finds AWS access keys in git history; `AKIAIOSFODNN7EXAMPLE` visible in a commit
Root Cause	Developer hardcoded credentials in code or config file; no pre-commit hook to scan

Resolution:

```
# 1. Immediately rotate/delete the exposed key (see Issue 9.2)

# 2. Remove from git history using BFG Repo Cleaner
bfg --replace-text secrets.txt my-repo.git
git reflog expire --expire=now --all && git gc --prune=now --aggressive
git push --force  # After coordinating with team

# 3. Install pre-commit hooks to prevent future leaks
pip install pre-commit
cat > .pre-commit-config.yaml << 'EOF'
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.2
    hooks:
      - id: gitleaks
EOF
pre-commit install

# 4. Add to CI pipeline as first step (fail fast)
```

```
  - name: Gitleaks Scan
    uses: gitleaks/gitleaks-action@v2
    # Fails pipeline if any secret pattern is found
```

Issue 14.2 — Trivy container scan blocking deployments with false positives

Attribute	Detail
Layer	DevSecOps / Container Scanning
Symptom	Trivy reports CRITICAL CVEs that are either non-exploitable or already mitigated; pipeline blocked on every merge
Root Cause	CVE in a package that is present but not used by the application; NVD data may have incorrect CVSS scores

Resolution — Tiered approach:

```
# CI pipeline: Hard-fail only on exploitable CRITICAL
- name: Trivy Scan
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${{ env.IMAGE_URI }}
    format: sarif
    output: trivy-results.sarif
    severity: CRITICAL,HIGH
    exit-code: '1'           # Hard fail
    ignore-unfixed: true     # Ignore CVEs with no patch yet (can't fix them
anyway)
    # Add specific CVE exemptions with justification:
    trivyignores: .trivyignore

# .trivyignore file (with justification comments):
# CVE-2023-XXXXX  # Not exploitable: package used for X, not Y
# CVE-2023-YYYYY  # Fix not yet available; mitigated by: [describe
mitigation]
```

Process: Review CRITICAL findings: if no fix available AND not exploitable in your context → add to `.trivyignore` with business justification + expiry date for re-review.

Issue 14.3 — SonarCloud quality gate blocking PRs on coverage threshold

Attribute	Detail
Layer	DevSecOps / SAST
Symptom	PRs blocked because test coverage drops below 80% threshold; developers bypassing quality gate
Root Cause	Coverage calculated on entire codebase including generated code, config files; threshold too aggressive for new projects

Resolution:

```
# sonar-project.properties — exclude generated/config code from coverage
sonar.coverage.exclusions=**/generated/**,**/config/**,**/*Config.java,**/*A
pplication.java
sonar.exclusions=**/test/**,**/target/**

# In CI — set correct coverage report path
sonar.java.coveragePlugin=jacoco
sonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml

# Quality gate strategy: soft fail with reporting (not hard block) during
ramp-up
# sonar.qualitygate.wait=true → blocks; sonar.qualitygate.wait=false →
reports but doesn't block
```

Issue 14.4 — WAF blocking legitimate traffic (false positives)

Attribute	Detail
Layer	WAF / Security
Symptom	Customers report 403 errors on legitimate requests; API calls from automated clients blocked

Attribute	Detail
Root Cause	WAF Managed Rules matching on benign request patterns (e.g., `AWSManagedRulesCommonRuleSet` blocking URLs with certain query parameter patterns)

Resolution — Staged WAF deployment:

```
Step 1: Deploy WAF rules in COUNT mode (not BLOCK)
  → Logs all matches but doesn't block traffic
  → Monitor CloudWatch WAF metrics and sample blocked requests

Step 2: Analyze sampled requests
  → WAF console → Sampled requests → identify false positive patterns

Step 3: Add exceptions for legitimate patterns
  aws wafv2 create-rule-group with custom rules that ALLOW before managed
rules run

Step 4: Switch managed rules to BLOCK mode after false positive tuning

Step 5: Regularly review:
  # Check WAF rule match counts
  aws cloudwatch get-metric-data --metric-data-queries '[
    {"Id":"waf","Expression":"SELECT AVG(BlockedRequests) FROM AWS/WAFV2
GROUP BY Rule"}
  ]'
```

Issue 14.5 — IMDSv1 SSRF attack on EC2 / EKS nodes

Attribute	Detail
Layer	EC2 Security / IMDSv2
Symptom	Application with SSRF vulnerability allows attacker to access `http://169.254.169.254/latest/meta-data/iam/security-credentials/` and steal node credentials
Root Cause	IMDSv1 is enabled — allows any GET request to retrieve credentials without session token (SSRF attacks can make GET requests through redirect chains)

Resolution:

```
# Enforce IMDSv2 on existing instances (require PUT session token)
aws ec2 modify-instance-metadata-options \
  --instance-id i-xxx \
  --http-tokens required \          # Forces IMDSv2
  --http-put-response-hop-limit 1   # Only local process, not forwarded

# Enforce via Launch Template (EKS node groups)
resource "aws_launch_template" "nodes" {
  metadata_options {
    http_tokens                 = "required"    # IMDSv2
    http_put_response_hop_limit = 1
    http_endpoint               = "enabled"
  }
}

# Account-level enforcement via AWS Config Rule
aws configservice put-config-rule --config-rule '{
  "Source": {"Owner": "AWS", "SourceIdentifier": "EC2_IMDSV2_REQUIRED"}
}'
```

---

15. Observability & Monitoring Issues

Issue 15.1 — Alert fatigue — too many non-actionable alerts

Attribute	Detail
Layer	Observability / SRE
Symptom	On-call team gets 50+ alerts per shift; team starts ignoring alerts ("alert fatigue"); real incidents missed
Root Cause	Alerting on causes (CPU high) instead of symptoms (users seeing errors); no alert ownership; no runbooks; too-sensitive thresholds

Resolution — Google's Alerting Principles:

```
Rules for every alert:
  1. Must be ACTIONABLE — "restart pod X" not "CPU is high"
  2. Must be URGENT — if it can wait until morning, it's not an alert (it's
a ticket)
```

```
  3. Must have a RUNBOOK linked in the annotation
  4. Alert on SYMPTOMS, not causes:
     ✅ "Error rate > 1% for 5 minutes" (user-visible symptom)
     ❌ "CPU > 80%" (cause — may not affect users)

Multi-window burn rate alerting (better than simple threshold):
  # Page if error budget burning 14x fast for 1 hour AND 5x fast for 6 hours
  # This catches both fast burns (acute incident) and slow burns (gradual
degradation)
  for: 1h
  expr: |
    (
      rate(http_requests_total{status=~"5.."}[1h])
      / rate(http_requests_total[1h])
    ) > 14 * 0.001  # 14x burn rate against 0.1% error budget
```

Monthly Alert Review: For each alert: "Did anyone act on it? Was the action documented?" If "no action taken" → delete or downgrade to warning.

Issue 15.2 — CloudWatch log insights queries too slow / expensive

Attribute	Detail
Layer	CloudWatch / Observability
Symptom	Log Insights queries timeout or return partial results; per-query costs adding up
Root Cause	Querying across too many log groups; no log level filtering; no structured logging (plaintext is harder and slower to query)

Resolution:

```
-- Expensive (scans all fields):
fields @timestamp, @message | filter @message like /ERROR/

-- Optimized (parse structured JSON logs):
fields @timestamp, level, service, message, trace_id
| filter level = "ERROR"
| filter service = "payment-service"
| sort @timestamp desc
| limit 100

-- Use metric filters instead of querying for known patterns:
aws logs put-metric-filter \
```

```
  --log-group-name /app/payment-service \
  --filter-name PaymentErrors \
  --filter-pattern '{ $.level = "ERROR" && $.service = "payment-service" }'
\
  --metric-transformations
metricName=PaymentErrors,metricNamespace=App,metricValue=1
```

Cost Control: Set log retention policies (7 days for dev, 30 days for prod, 90 days for audit) to avoid unbounded log storage costs.

Issue 15.3 — Pod metrics missing from Prometheus (targets showing DOWN)

Attribute	Detail
Layer	Prometheus / Kubernetes Observability
Symptom	Prometheus targets page shows service as DOWN; no metrics visible in Grafana
Root Cause	(a) ServiceMonitor selector not matching service labels; (b) Pod not exposing `/metrics` endpoint; (c) NetworkPolicy blocking Prometheus scrape

Resolution:

```
# 1. Check Prometheus targets
kubectl port-forward -n monitoring svc/prometheus-operated 9090:9090
# Browse http://localhost:9090/targets → Find the failing target

# 2. Verify ServiceMonitor label selector matches Service labels
kubectl get servicemonitor my-app -o yaml
# spec.selector.matchLabels must match labels on the Service

# 3. Test metrics endpoint directly
kubectl exec -it <prometheus-pod> -n monitoring -- \
  wget -qO- http://my-app-service.my-
namespace.svc.cluster.local:8080/metrics | head -20

# 4. NetworkPolicy — allow Prometheus namespace to scrape
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-prometheus-scrape
  namespace: my-namespace
```

```
spec:
  podSelector:
    matchLabels:
      app: my-app
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: monitoring
    ports:
    - port: 8080
EOF
```

---

16. Cost & FinOps Issues

Issue 16.1 — Unexpected AWS bill spike

Attribute	Detail
Layer	FinOps / Cost Management
Symptom	Monthly AWS bill 50% higher than expected; unclear what drove the increase
Root Cause	(a) NAT Gateway data transfer charges from unoptimized S3 access; (b) EC2 instances not stopped in dev environment; (c) Cross-AZ data transfer; (d) Forgotten load balancer or NAT Gateway

Investigation:

```
# 1. Use Cost Explorer to find the spike
aws ce get-cost-and-usage \
  --time-period Start=2026-06-01,End=2026-06-16 \
  --granularity DAILY \
  --metrics UnblendedCost \
  --group-by Type=DIMENSION,Key=SERVICE

# 2. Cost by resource (requires Cost Allocation Tags)
aws ce get-cost-and-usage \
  --group-by Type=TAG,Key=Environment \
  # Shows cost breakdown by Environment=prod/dev/staging

# 3. Find untagged/orphaned resources
aws resourcegroupstaggingapi get-resources \
```

```
  --resource-type-filters ec2:instance,elasticloadbalancing:loadbalancer \
  --tag-filters Key=Environment  # Resources WITHOUT this tag are untagged
```

Common Cost Reduction Actions:

Issue	Fix	Saving
S3 traffic via NAT	Add S3 Gateway VPC Endpoint (FREE)	Eliminates NAT charges for S3
Dev running 24/7	Lambda + EventBridge auto-shutdown at 7 PM	Save 65% compute
gp2 EBS volumes	Migrate to gp3 (same IOPS, lower cost)	20% storage savings
Right-size EC2	AWS Compute Optimizer recommendations	20-40% compute savings
Single-AZ NAT GW	Keep single NAT per AZ (not multiple per AZ)	Avoid duplicate charges

Issue 16.2 — Reserved Instance underutilized (paying for unused reservation)

Attribute	Detail
Layer	FinOps / Reserved Instances
Symptom	AWS Cost Explorer shows RI utilization < 80%; paying for RIs but instances are off or different type
Root Cause	Workload changed; instances were resized or terminated; RI purchased for wrong instance type

Resolution:

```
# Check RI utilization
aws ce get-reservation-utilization \
  --time-period Start=2026-06-01,End=2026-06-16

# Options:
# 1. Sell unused RIs on AWS Marketplace (standard/convertible)
```

```
# 2. Modify RI to matching instance type (convertible RIs only)
# 3. Use "Savings Plans" instead — more flexible than RIs
#    Compute Savings Plans apply to any EC2 instance type, size, AZ, OS,
tenancy
#    More forgiving when workloads change

# Best practice: Compute Savings Plans > EC2 Instance RIs (for flexibility)
aws savingsplans list-savings-plans \
  --query 'savingsPlans[].
{type:savingsPlanType,state:state,utilization:recurringPaymentAmount}'
```

---

17. Disaster Recovery & High Availability Issues

Issue 17.1 — Route 53 failover not routing to DR during primary region outage

Attribute	Detail
Layer	Route 53 / DR
Symptom	Primary region ALB becomes unavailable; DNS still resolves to primary (no failover); users get errors
Root Cause	Route 53 health check not configured on primary record; or health check checks the wrong endpoint; or health check failing but TTL hasn't expired for cached DNS

Resolution:

```
# Route 53 Failover with proper health checks
resource "aws_route53_health_check" "primary" {
  fqdn              = aws_lb.primary.dns_name
  port              = 443
  type              = "HTTPS"
  resource_path     = "/health"
  failure_threshold = 3
  request_interval  = 10   # Check every 10s
}

resource "aws_route53_record" "api_primary" {
  zone_id         = var.zone_id
  name            = "api.company.com"
```

```
    type            = "A"
    set_identifier  = "primary"
    health_check_id = aws_route53_health_check.primary.id

    failover_routing_policy {
      type = "PRIMARY"
    }
    alias {
      name                   = aws_lb.primary.dns_name
      zone_id                = aws_lb.primary.zone_id
      evaluate_target_health = true
    }
  }

  resource "aws_route53_record" "api_secondary" {
    zone_id        = var.zone_id
    name           = "api.company.com"
    type           = "A"
    set_identifier = "secondary"

    failover_routing_policy {
      type = "SECONDARY"
    }
    alias {
      name                   = aws_lb.secondary.dns_name
      zone_id                = aws_lb.secondary.zone_id
      evaluate_target_health = true
    }
  }
```

DNS TTL consideration: Set low TTL (60s) on failover records so DNS propagation is fast during failover. Clients respect TTL — lower TTL = faster failover but more DNS queries.

Issue 17.2 — RDS Multi-AZ failover takes too long (> 2 minutes)

Attribute	Detail
Layer	RDS / Reliability
Symptom	During AZ failure, RDS Multi-AZ failover takes 60-120 seconds; application error budget consumed
Root Cause	Standard RDS Multi-AZ uses crash recovery on the standby (~60s); not truly hot standby

Resolution — Use Aurora instead of RDS Multi-AZ for critical workloads:

Feature	RDS Multi-AZ	Aurora Multi-AZ
Failover time	60-120 seconds	< 30 seconds
Read replicas	5 (separate)	Up to 15 (same cluster)
Cross-region	Manual snapshot	Aurora Global Database
RPO	Near-zero (synchronous)	Near-zero (synchronous within region)
Global RTO	N/A (manual)	< 1 minute (Aurora Global)

For existing RDS Multi-AZ: Add application-level retry logic with exponential backoff to handle the 60-120s failover window gracefully.

Issue 17.3 — DR tested but RTO target not met during real outage

Attribute	Detail
Layer	DR / Business Continuity
Symptom	Real outage occurred; DR procedures documented but took 4x longer than estimated
Root Cause	DR runbooks not tested recently; manual steps took longer under pressure; dependencies not pre-warmed

Resolution — Automate DR failover:

```
# AWS Step Functions DR Automation Workflow
states = {
  "VerifyOutage": check_primary_health(),
  "PromoteAurora":
aws_rds.failover_global_cluster(GlobalClusterIdentifier="prod-global"),
  "ScaleEKS_DR": aws_eks.scale_node_group(cluster="dr-cluster", desired=6),
  "UpdateRoute53": aws_route53.change_failover_to_secondary(),
  "VerifyDRHealth": check_dr_health(),
  "NotifyTeam": aws_sns.publish(TopicArn="incident-notifications")
}
```

DR Testing Cadence:

- Quarterly: Full DR failover test (use AWS FIS — Fault Injection Simulator)
- Monthly: Verify RTO/RPO metrics are still achievable
- Annual: Full game day simulation including communications

---

18. Multi-Account & Landing Zone Issues

Issue 18.1 — Developers accidentally spinning up expensive resources in prod account

Attribute	Detail
Layer	AWS Organizations / SCPs
Symptom	GPU instances, large RDS, or unnecessary resources created in production account without approval
Root Cause	No Service Control Policies preventing unapproved resource types; developers have PowerUser access to production

Resolution — SCPs for resource governance:

```
// Deny expensive instance families in non-prod accounts:
{
  "Effect": "Deny",
  "Action": "ec2:RunInstances",
  "Resource": "arn:aws:ec2:*:*:instance/*",
  "Condition": {
    "ForAnyValue:StringLike": {
      "ec2:InstanceType": ["p3.*", "p4.*", "g4.*", "g5.*", "x1.*", "u-*"]
    }
  }
}

// Deny creating resources outside approved regions:
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
```

```
      "aws:RequestedRegion": ["us-east-1", "us-west-2", "ap-south-1"]
    },
    "BoolIfExists": {
      "aws:PrincipalIsAWSService": "false"
    }
  }
}
```

Issue 18.2 — CloudTrail audit logs missing for some accounts

Attribute	Detail
Layer	Multi-Account / Compliance
Symptom	Compliance audit finds gaps in CloudTrail; some accounts have no logging or logs in the account (not centralized)
Root Cause	CloudTrail not enabled at organization level; SCP not preventing trail deletion; new accounts joined without inheriting controls

Resolution:

```
# Enable organization-wide CloudTrail (cannot be disabled by member
accounts)
aws cloudtrail create-trail \
  --name org-cloudtrail \
  --s3-bucket-name centralized-cloudtrail-logs \
  --is-organization-trail \          # Covers all member accounts
  --include-global-service-events \
  --is-multi-region-trail             # All regions

# SCP to prevent trail deletion/disabling
{
  "Effect": "Deny",
  "Action": [
    "cloudtrail:DeleteTrail",
    "cloudtrail:StopLogging",
    "cloudtrail:UpdateTrail"
  ],
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {"aws:PrincipalArn":
"arn:aws:iam::LOG_ARCHIVE_ACCOUNT:role/CloudTrailAdmin"}
  }
}
```

Issue 18.3 — New account provisioned without security baselines

Attribute	Detail
Layer	Landing Zone / Account Vending
Symptom	New AWS account created manually; GuardDuty, Security Hub, Config not enabled; no VPC flow logs
Root Cause	No automated account vending process; manual account creation skips baseline configurations

Resolution — Account Factory Automation:

```
# AWS Control Tower Account Factory (Terraform) or custom Lambda
def provision_account(account_name, email, ou_id):
    # 1. Create account via Organizations
    account = aws_organizations.create_account(Name=account_name,
Email=email)

    # 2. Move to correct OU
    aws_organizations.move_account(AccountId=account.id,
DestinationParentId=ou_id)

    # 3. Assume role in new account and apply baseline
    credentials = assume_cross_account_role(account.id,
"OrganizationAccountAccessRole")
    baseline_session = boto3.Session(**credentials)

    # 4. Enable security services
    baseline_session.client("guardduty").create_detector(Enable=True)
    baseline_session.client("securityhub").enable_security_hub()
    baseline_session.client("config").put_configuration_recorder(...)

    # 5. Apply baseline Terraform (networking, IAM, tagging)
    run_terraform(env=account.id, module="account-baseline")
```

19. SRE / Incident Response Issues

Issue 19.1 — Incident response is slow (MTTR > 1 hour for SEV1)

Attribute	Detail
Layer	SRE / Operations
Symptom	Production outages take 1-2 hours to resolve; on-call team spends too long investigating
Root Cause	No runbooks linked to alerts; on-call doesn't know what to do; alerts don't point to the symptom; no escalation path

Resolution Framework:

```
Every alert must have:
1. RUNBOOK LINK in alert annotation:
   annotations:
     runbook: https://wiki.company.com/runbooks/payment-service-high-error-
rate

2. RUNBOOK must include:
   - What this alert means
   - Immediate mitigation (not root cause — fix first, debug later)
   - Escalation path if mitigation doesn't work
   - Links to dashboards

3. ESCALATION path:
   Primary on-call → no response in 5 min → Secondary on-call
   Secondary on-call → no response in 5 min → Engineering Manager
   EM → no resolution in 30 min → VP Engineering

4. STATUS PAGE update within 10 minutes of SEV1 declaration

5. BLAMELESS POSTMORTEM within 48 hours
```

Issue 19.2 — Database lock contention causing production incident

Attribute	Detail
Layer	Database / Incidents

Attribute	Detail
Root Cause	`ALTER TABLE` migration ran on hot table without online schema change tool → locked all writes → cascade 504s
Mitigation	Rollback migration; restart connection pool

Prevention — Safe migration practices:

```
# Never: ALTER TABLE on production hot table (locks entire table)
# ALTER TABLE orders ADD COLUMN discount_pct DECIMAL(5,2);

# Instead: Use pt-online-schema-change (MySQL) or pg_repack (PostgreSQL)
# pt-osc creates a shadow table, copies data, swaps atomically:
pt-online-schema-change \
  --alter "ADD COLUMN discount_pct DECIMAL(5,2)" \
  D=appdb,t=orders \
  --execute --no-drop-old-table

# Or for PostgreSQL: Use concurrently for indexes
CREATE INDEX CONCURRENTLY idx_orders_discount ON orders(discount_pct);
# "CONCURRENTLY" allows reads/writes during index build

# Migration review checklist must include:
# ✦ Does this lock the table?
# ✦ Is this reversible?
# ✦ Has it been tested with production-sized data in staging?
# ✦ Is there a rollback script?
```

Issue 19.3 — Error budget exhausted — what to do?

Attribute	Detail
Layer	SRE / Error Budgets
Symptom	Monthly error budget (e.g., 43 min for 99.9% SLO) consumed with 2 weeks remaining

Response Protocol:

```
Error budget remaining:
  > 50%  → Ship features normally, take calculated risks
  20–50% → Code freeze on risky changes, increase testing, review deployment
```

```
process
  < 20%  → Feature freeze, all hands on reliability
  0%     → All engineering shifted to reliability work; no new features
until next period

Actions when budget < 20%:
1. Identify top reliability risks (from postmortems)
2. Create reliability sprint focusing on:
   - Fixing known flaky tests/services
   - Improving alerting precision
   - Completing deferred runbook updates
   - Load testing and capacity planning
3. Present findings + fix plan to product management
4. Document in error budget policy (signed by product + engineering)
```

---

20. Quick Reference — Troubleshooting Commands

AWS Networking Debug

```
# VPC Reachability Analyzer (traces path between resources)
aws ec2 create-network-insights-path \
  --source <instance-id> --destination <rds-endpoint-id> \
  --protocol tcp --destination-port 5432

# VPC Flow Logs query (CloudWatch Logs Insights)
fields @timestamp, srcAddr, dstAddr, dstPort, action, protocol
| filter action = "REJECT"
| filter dstPort = 5432
| sort @timestamp desc
| limit 50

# Check security group rules for an instance
aws ec2 describe-instance-attribute --instance-id i-xxx --attribute groupSet

# Check effective NACL rules
aws ec2 describe-network-acls --filter Name=association.subnet-
id,Values=subnet-xxx
```

EKS / Kubernetes Debug

```
# Pod debugging
kubectl describe pod <pod> -n <ns>     # Events, conditions, resource limits
kubectl logs <pod> -n <ns> --previous  # Previous container logs (CrashLoop)
kubectl exec -it <pod> -n <ns> -- sh   # Interactive shell
kubectl top pods --all-namespaces      # Resource usage

# Cluster-wide health
kubectl get nodes -o wide
kubectl get pods --all-namespaces | grep -Ev 'Running|Completed'
kubectl get events --sort-by='.lastTimestamp' -A | tail -30

# Network debug
kubectl run debug --image=nicolaka/netshoot --rm -it -- bash

# Aurora connectivity from pod
kubectl run db-test --image=postgres:15-alpine --rm -it -- \
  psql -h <aurora-endpoint> -U user -d db -c "SELECT 1;"
```

IAM Debug

```
# Simulate IAM permissions
aws iam simulate-principal-policy \
  --policy-source-arn <role-arn> \
  --action-names s3:GetObject \
  --resource-arns arn:aws:s3:::my-bucket/*

# Check who last used a role
aws iam get-role --role-name my-role \
  --query 'Role.RoleLastUsed'

# List recent API calls for investigation
aws cloudtrail lookup-events \
  --start-time $(date -v-1H +%Y-%m-%dT%H:%M:%S) \
  --lookup-attributes AttributeKey=EventName,AttributeValue=AssumeRole
```

Terraform Debug

```
terraform plan -detailed-exitcode   # Exit 2 = changes needed
terraform state list                # All tracked resources
terraform state show <resource>     # Details of a specific resource
terraform state mv <old> <new>      # Rename without destroying
terraform import <resource> <id>    # Import existing AWS resource
terraform force-unlock <lock-id>    # Release stuck state lock
TF_LOG=DEBUG terraform plan         # Verbose debug output
```

Cost Analysis

```
# Daily spend by service
aws ce get-cost-and-usage \
  --time-period Start=2026-06-01,End=2026-06-16 \
  --granularity DAILY \
  --metrics UnblendedCost \
  --group-by Type=DIMENSION,Key=SERVICE

# Identify untagged resources
aws resourcegroupstaggingapi get-resources \
  --resources-per-page 50 \
  | jq '.ResourceTagMappingList[] | select(.Tags | length == 0) |
.ResourceARN'
```

Key Interview Principles to Internalize

Principle	Application
Fail fast on cheap checks	Gitleaks before SAST before Docker build in CI
Mitigate first, debug later	In incidents: restore service before root-causing
Explicit deny wins	In IAM: a single Deny overrides all Allows
Stateful vs. Stateless	SG = stateful (return auto-allowed); NACL = stateless (must allow both directions)
Reference SG IDs, not CIDRs	Internal security group rules should reference SG IDs
IMDSv2 always	Prevents SSRF-based credential theft from EC2 metadata
Use SHA tags, not `latest`	Immutable, traceable, ArgoCD-compatible image tags
Error budget as API	When budget is exhausted, features stop; reliability work starts
Blameless postmortems	Focus on systems, not people; learning culture drives reliability

Principle	Application
Private subnets for everything compute	Only ALBs and NAT GWs go in public subnets
OIDC over access keys	No long-lived secrets in CI/CD; OIDC tokens auto-expire in 15 min
Prevent_destroy for databases	Lifecycle block prevents accidental `terraform destroy`
VPC Endpoints reduce cost AND risk	S3 Gateway endpoint is free; eliminates NAT charges for S3
PDB before node drain	Always ensure PDBs exist before EKS upgrade to ensure zero-downtime

---

Prepared: June 2026 | Pushparaj Naik — AWS Cloud & DevOps Architect | 22+ Years
Scope: AWS Networking · Compute · EKS/K8s · DevOps · DevSecOps · IaC · SRE · FinOps